Soluciones: Funciones y arreglos

Programación I, Semana 8

Datos usados en todos los ejercicios:

```
const int N = 7;
int temp[N] = {12, 15, 9, 17, 14, 8, 16};
//   índice:    0   1  2   3   4  5   6
```

Ejercicios obligatorios

Ejercicio 1: La temperatura más baja

```
int minimo(int a[], int n) {
    int min = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] < min) {
            min = a[i];
        }
    }
    return min;
}
```

Por qué: se empieza suponiendo que el menor es el primero (`a[0]`). Luego se revisa el resto: si aparece uno más pequeño, se guarda.
También es correcto: empezar el `for` en `i = 0`. Solo hace una comparación de más.
Errores frecuentes:

- `int min = 0;`: con temperaturas positivas daría 0, un valor que no está en el arreglo.
- `i <= n`: el último índice válido es `n - 1`.
- Dejar `>` en vez de `<`: eso calcula el máximo.

Ejercicio 2: Seguimiento

`i`	`a[i]`	`¿a[i] < min?`	`min`
inicio			12
1	15	no	12
2	9	sí	9
3	17	no	9
4	14	no	9

i	a[i]	¿a[i] < min?	min
5	8	sí	8
6	16	no	8

La función devuelve **8**.
Revisar: la columna `min` solo cambia cuando la respuesta es «sí». Si el ciclo empieza en `i = 0`, hay una fila más (`i = 0`, `12 < 12`: no) y también está bien.

Ejercicio 3: Días calurosos

```
int contarMayoresQue(int a[], int n, int umbral) {
    int c = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > umbral) {
            c++;
        }
    }
    return c;
}
```

Línea de `main`:

```
std::cout << contarMayoresQue(temp, N, 13);
```

Los mayores que 13 son 15, 17, 14 y 16: devuelve **4**.
Errores frecuentes:

- `c = c + a[i];`: eso es un **acumulador**: suma las temperaturas. Aquí se necesita un **contador**: sumar 1 cada vez.
- `a[i] >= umbral`: «mayores que» no incluye al 13.
- Llamar con `temp[]` o `temp[N]`: al llamar se escribe el nombre del arreglo **sin corchetes**.

Ejercicio 4: ¿Qué imprime?

a) Imprime `5 0 8`.

- `t` sigue valiendo 5: la función recibió una **copia** del entero y cambió la copia.
- `v[0]` ahora vale 0: un arreglo **no** se copia; la función modifica el mismo arreglo de `main`.
- `v[1]` vale 8: nadie lo modificó.

b) Imprime `10`.

- `doble(3)` es 6.
- `doble(doble(1))`: primero `doble(1)` es 2, y luego `doble(2)` es 4.
- $6 + 4 = 10$.

Ejercicios intermedios

Ejercicio 5: Promedio de la semana

```
double promedio(int a[], int n) {
    int suma = 0;
    for (int i = 0; i < n; i++) {
        suma = suma + a[i];
    }
    return static_cast<double>(suma) / n;
}
```

La suma es 91 y 91 / 7 = **13**.
Atención: `return suma / n;` con estos datos también da 13, pero **está mal**: con otras temperaturas perdería los decimales. Por ejemplo, si la suma fuera 90, daría 12 en lugar de 12.857. Hay que convertir a `double` antes de dividir. También es correcto `suma * 1.0 / n`.

Ejercicio 6: Encontrar los errores

N.º	Error	Corrección
1	`int max = 0;`: falla si todas las temperaturas son negativas.	`int max = a[0];`
2	`i <= n`: en la última vuelta lee `a[n]`, fuera del arreglo.	`i < n`
3	La función dice que devuelve `int`, pero no tiene `return`.	Agregar `return max;` antes de la `}` final.
4	`maximo(temp[], N)`: al llamar no se usan corchetes.	`maximo(temp, N)`

Código corregido:

```
int maximo(int a[], int n) {
    int max = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }
    return max;
}
// en main:
std::cout << maximo(temp, N);    // imprime 17
```

Reto

Ejercicio 7: El día más frío

a)

```
int posicionDelMinimo(int a[], int n) {
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] < a[pos]) {
            pos = i;
        }
    }
    return pos;
}
```

Devuelve **5**, porque `temp[5]` es 8. La idea: en vez de guardar el valor más pequeño, se guarda **dónde** está. Se compara con `a[pos]`, el menor encontrado hasta ahora.

b)

```
double p = promedio(temp, N);
int frios = 0;
for (int i = 0; i < N; i++) {
    if (temp[i] < p) {
        frios++;
    }
}
std::cout << "Dias mas frios que el promedio: " << frios;
```

El promedio es 13. Los días más fríos son 12, 9 y 8: imprime **3**.

Detalle: conviene guardar el promedio en una variable antes del ciclo, para no calcularlo de nuevo en cada vuelta.